

Отчёт по третьему этапу внешнего курса Stepik

Продвинутые темы

Иванова Ангелина Олеговна

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Выполнение 3.1. Текстовый редактор vim	8
3.2	Выполнение 3.2. Скрипты на bash: основы	11
3.3	Выполнение 3.3. Скрипты на bash: ветвления и циклы	14
3.4	Выполнение 3.4. Скрипты на bash: разное	21
3.5	Выполнение 3.5. Продвинутый поиск и редактирование	28
3.6	Выполнение 3.6. Строим графики в gnuplot	32
3.7	Выполнение 3.7. Разное	35
4	Выводы	40
5	Список литературы	41

Список иллюстраций

3.1	Выход из редактора vim	8
3.2	Использование w и W	9
3.3	Изменение строки	10
3.4	Замена Windows на Linux	10
3.5	Режим Visual в vim	11
3.6	История команд при вложенных оболочках	12
3.7	Абсолютный путь созданного файла	12
3.8	Допустимые имена переменных	13
3.9	Скрипт вывода аргументов	13
3.10	Всегда истинные условия	15
3.11	Ветвление с числовыми сравнениями	16
3.12	Скрипт для подсчёта студентов	17
3.13	Цикл с continue	18
3.14	Скрипт возрастной группы – условие	19
3.15	Скрипт возрастной группы – решение	19
3.16	Увеличение переменной	21
3.17	Вывод текущего каталога в скрипте	22
3.18	Возврат внешней программы	22
3.19	Локальные переменные в функции	23
3.20	Задание на НОД	24
3.21	Решение НОД	24
3.22	Задание на калькулятор	26
3.23	Решение калькулятора	26
3.24	find с -iname и -name	28
3.25	Сравнение -path и -name	29
3.26	mindepth и maxdepth в find	29
3.27	сравнение размеров вывода grep с контекстом	30
3.28	grep с регулярным выражением	30
3.29	sed без -n	31
3.30	Замена аббревиатур – условие	31
3.31	Замена аббревиатур – решение	32
3.32	Сохранение графиков после закрытия gnuplot	32
3.33	autotitle columnhead без заголовка	33
3.34	Установка пользовательских xtics	34
3.35	Модифицированный move.rot	35
3.36	Изменение прав доступа	36

3.37	Предоставление доступа к чужой директории	37
3.38	Возможности wc	38
3.39	Вывод размера директории	38
3.40	Кратчайшее создание нескольких каталогов	39

Список таблиц

1 Цель работы

Целью данной работы является выполнение внешнего курса под названием «Введение в Linux». В третьем этапе мы подробно редактор vim, скрипты bash и различные возможности Linux.

2 Задание

1. Ознакомиться с теоретическим материалом
2. Ответить на вопросы и выполнить задания для закрепления теоретического материала

3 Выполнение лабораторной работы

3.1 Выполнение 3.1. Текстовый редактор vim

Чтобы выйти из редактора vim необходимо нажать ” : «, затем»q”, затем «Enter». Именно так осуществляется выход из нормального режима vim (рис. 3.1)

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

Вы прошли больше 80% курса, оставьте отзыв [Оставить отзыв](#) [Нет, спасибо](#)

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

✔ Отличное решение!

- ☐ "Esc"
- ☐ "Q"
- ☐ "q", затем "Enter"
- ☐ ":", затем "q"
- ☒ ":", затем "q", затем "Enter"

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл [Мои решения](#)

Верно решили 47 439 учащихся
Из всех попыток 67% верных

Рисунок 3.1: Выход из редактора vim

Далее необходимо было изучить разнице между использованием «W» и «w» в редакторе vim. W служит перемещению между условно называемых больших слов - слова разделенные исключительно пробелом. Использование w служит для перемещению между условно называемых маленьких слов - слова делятся всеми

символами, кроме нижнего подчеркивания, при этом символы тоже считаются маленькими словами. На примере строки «Strange_ TEXT is_here. 2=2 YES!» мы можем увидеть, что больших слов здесь всего 5 («Strange_», «TEXT», «is_here.», «2=2», «YES!»), а маленьких 9 («Strange_», «TEXT», «is_here», «.», «2», «=», «2», «YES», «!»). Логически выбрали правильные ответы: «Нажимая только на W, нельзя переместить курсор на».” «,»После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w”, «Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w» (рис. 3.2).

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

При перемещении в vim "по словам" есть небольшая разница в том, используем мы маленькую (w, e, b) или большую (W, E, B) букву. Первые перемещают нас по "словам" (word), а вторые по "большим словам" (WORD). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между word и WORD.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:
Strange_ TEXT is_here. 2=2 YES!

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению **w**, нужно открыть vim и ввести команду **:help w**. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про **e** и про **b** и, самое главное, про word и WORD. Кроме того, можно вызвать сразу справку по термину word при помощи **:help word**. Чтобы закрыть справку, нужно ввести команду **:q**.

Выберите все подходящие ответы из списка

☒ Все правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Нажимая только на W, нельзя переместить курсор на "."
- ☒ После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w
- ☐ Чтобы попасть в конец строки, нужно одинаковое число нажатий, что на W, что на w
- ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w
- ☐ В этой строке 12 "слов" (word)
- ☐ Нажимая только на w, нельзя переместить курсор на "."

[Следующий шаг](#) [Решить снова](#)

Рисунок 3.2: Использование w и W

В текстовом файле записана одна единственная строка: «one two three four five» и нужно преобразовать её в строку «three four four four five». Для этого подходят команды: «ddthree four four four five<>» (удаление всей строки и вставка нового текста), «d2wwyWPr» (удаление двух первых слов, переход к слову «four», его копирование и вставка перед и после курсора), «d2w\$\$\$bifour four <>» (удаление двух слов, переход в конец строки и вставка «four four » перед последним словом), «d2wwifour four <>» (удаление двух слов, переход к «four» и вставка перед ним) (рис. 3.3).

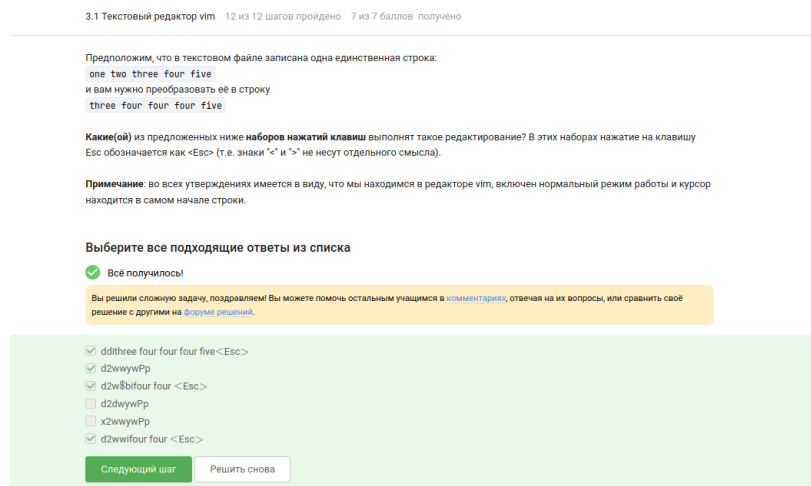


Рисунок 3.3: Изменение строки

Чтобы в редакторе vim заменить в некотором файле все строки, содержащие слово Windows, на такие же строки, но со словом Linux, при этом если в какой-то строке слово Windows встречается больше, чем один раз, то заменить на Linux в этой строке нужно только самое первое из этих слов нужно выполнить команду «:%s/Windows/Linux». Без дополнительных флагов замена в vim затрагивает только первое совпадение на строке (рис. 3.4).

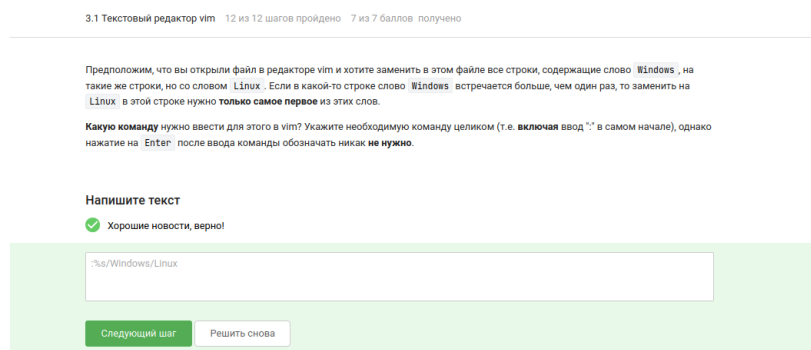


Рисунок 3.4: Замена Windows на Linux

При самостоятельном изучении режима выделения были отмечены верные утверждения: «Режим выделения открывается из нормального режима по нажатию»v»«,»В режиме выделения можно использовать команды d (удалить) и y (скопировать)«,»В режиме выделения можно использовать команды перемещения

(например, W, e, \$, и др.)» Выйти из режима выделения можно, нажав клавишу Esc два раза” (рис. 3.5).

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

✔ Все правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Режим выделения открывается из нормального режима по нажатию "v"
- ☐ Режим выделения открывается из любого другого режима по нажатию "V"
- ☒ В режиме выделения можно использовать команды d (удалить) и u (скопировать)
- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза

Следующий шаг Решить снова

Рисунок 3.5: Режим Visual в vim

3.2 Выполнение 3.2. Скрипты на bash: основы

Если из bash запущена sh, а из неё снова bash, то стрелки вверх/вниз показывают только команды, набранные в текущей (последней) оболочке. Поэтому доступен только набор C (рис. 3.6).

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

✓ Всё получилось!

- ☐ Из наборов B и C
- ☐ Только из набора B
- ☐ Никакие команды появляться не будут
- ☐ Из наборов A и C
- ☒ Только из набора C

Следующий шаг

Решить снова

Рисунок 3.6: История команд при вложенных оболочках

Скрипт создаёт файл в `«/home/bi/»`, поэтому абсолютный путь к `«file1.txt»` будет `«/home/bi/file1.txt»` (рис. 3.7).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть абсолютный путь до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

✓ Отличное решение!

- ☒ `/home/bi/file1.txt`
- ☐ Никкак (файла `file1.txt` не будет существовать после завершения работы скрипта)
- ☐ `/home/bi/Documents/file1.txt`
- ☐ `/home/bi/Desktop/file1.txt`

Следующий шаг

Решить снова

Рисунок 3.7: Абсолютный путь созданного файла

Имена переменных в `bash` могут содержать буквы (в том числе заглавные), цифры (не на первом месте) и знак подчёркивания. Поэтому из предложенного списка допустимыми являются `«VARiable»`, `«variable»`, `“__variable”`, `«variable_123»` и `«variable123»`. Строки `«var i able»` и `«123variable»` не удовлетворяют синтаксису (рис. 3.8).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

✔ Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ VARiable
- ☒ variable
- ☐ var l able
- ☒ _variable
- ☐ 123variable
- ☒ variable_123
- ☒ variable123

Следующий шаг

Решить снова

Рисунок 3.8: Допустимые имена переменных

Для вывода аргументов был написан скрипт, использующий «\$1» и «\$2». При запуске «./script.sh one two» он печатает «Arguments are: \$1=one \$2=two» (рис. 3.9).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [arguments.sh](#).

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✔ Всё получилось!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 var1=$1
3 var2=$2
4
5 echo "Arguments are: $1=$var1 $2=$var2"
```

Следующий шаг

Решить снова

Рисунок 3.9: Скрипт вывода аргументов

Листинг:

```
#!/bin/bash
var1=$1
var2=$2

echo "Arguments are: \$1=$var1 \$2=$var2"
```

3.3 Выполнение 3.3. Скрипты на bash: ветвления и циклы

Среди условий, всегда истинных вне зависимости от аргументов, были отмечены: «!(4 -le 3)» (логическое отрицание ложного утверждения), «\$# -ge 0» (число аргументов всегда неотрицательно), «-e \$0» (файл скрипта существует), «-n \$0» (имя скрипта не пусто), «-z» “ ” (пустая строка имеет нулевую длину) и «5 -ge 5». Все они гарантированно возвращают истину (рис. 3.10).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](#).

Предположим, вы пишете скрипт на bash и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете вписать вместо `"..."` (внутри `[[ ... ]]` и **не забудьте про пробелы** после `[[` и перед `]]`) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

✓ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `! (4 -le 3)`
- ☒ `$# -ge 0`
- ☒ `-e $0`
- ☒ `-n $0`
- ☒ `-z "`
- ☒ `5 -ge 5`

Следующий шаг

Решить снова

Рисунок 3.10: Всегда истинные условия

Фрагмент с «if-elif-else» при «var=3» и «var=5» оба раза попадает в ветку «else», выводя «four». Таким образом, на экране сначала «four», потом «four» (рис. 3.11).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [branching2.sh](#), [branching3.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

☒ Так точно!

- ☐ Сначала two, потом one
- ☐ Сначала four, потом one
- ☒ Сначала four, потом four
- ☐ Сначала one, потом two

Следующий шаг

Решить снова

Рисунок 3.11: Ветвление с числовыми сравнениями

Был реализован скрипт, который по числу студентов выводит правильную форму: 0 → «No students», 1 → «1 student», 2–4 → «N students», от 5 и больше → «A lot of students». Решение использовало цепочку «if-elif-else» (рис. 3.12).

Напишите скрипт на Bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. "-->" выводить не нужно.

Примечание б): в последней строке слово "lot" с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно **проверить вход** (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать **не нужно!**

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задач, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через `stdin → stdout`

✓ Хорошие новости, верно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2
3 if [[ $1 -eq 1 ]]; then
4     echo "$1 student"
5 elif [[ $1 -gt 1 && $1 -le 4 ]]; then
6     echo "$1 students"
7 elif [[ $1 -ge 5 ]]; then
8     echo "A lot of students"
9 else
10    echo "No students"
11 fi
12
```

Следующий шаг

Решить снова

Рисунок 3.12: Скрипт для подсчёта студентов

Листинг:

```
#!/bin/bash
```

```
if [[ $1 -eq 1 ]]; then
    echo "$1 student"
elif [[ $1 -gt 1 && $1 -le 4 ]]; then
    echo "$1 students"
elif [[ $1 -ge 5 ]]; then
    echo "A lot of students"
else
    echo "No students"
fi
```

В цикле «for str in a, b, c_d» слово «start» выводится пять раз (согласно логике задания), а «finish» - 4 (рис. 3.13).

3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
echo "start"
if [[ $str > "c" ]]
then
continue
fi
echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово "start", а сколько раз слово "finish"?

Выберите один вариант из списка

✓ Так точно!

- ☐ 5 раз "start" и ни разу "finish"
- ☐ 5 раз "start" и 2 раза "finish"
- ☒ 5 раз "start" и 4 раза "finish"
- ☐ 3 раза "start" и 3 раза "finish"

Следующий шаг Решить снова

Рисунок 3.13: Цикл с continue

Задача с бесконечным циклом опроса имени и возраста решена при помощи вложенных «while» и проверок на пустой ввод или нулевой возраст. Скрипт корректно обрабатывает граничные условия и выходит по команде «bye» (рис. 3.14).

Напишите скрипт на bash, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт должен вывести сообщение **"enter your name:"** и ждать от пользователя ввода имени (используйте `read`, чтобы прочитать его). Когда имя введено, то скрипт должен написать **"enter your age:"** и ждать ввода возраста (опять нужен `read`). Когда возраст введен, скрипт пишет на экран **"<Имя>, your group is <группа>"**, где **<группа>** определяется на основе возраста по следующим правилам:

- младше либо равно 16: **"child"**,
- от 17 до 25 (включительно): **"youth"**,
- старше 25: **"adult"**.

После этого скрипт опять выводит сообщение **"enter your name:"** и всё начинается по новой (бесконечный цикл!). Если в какой-то момент работы скрипта будет введено **пустое имя** или **возраст 0**, то скрипт должен написать на экран **"bye"** и закончить свою работу (выход из цикла!).

Примеры корректной работы скрипта:

№1

```
./script.sh
enter your name:
Egor
enter your age:
16
Egor, your group is child
enter your name:
Elena
enter your age:
0
bye
```

№2:

```
./script.sh
enter your name:
Elena Petrovna
enter your age:
25
Elena Petrovna, your group is youth
enter your name:

bye
```

Подсказка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Рисунок 3.14: Скрипт возрастной группы – условие

Напишите программу. Тестируется через stdin → stdout

✓ Здорово, всё верно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

```
1 child=16
2 adult=25
3 stdout=0
4
5 while [[ $stdout != 1 ]]
6 do
7     echo "enter your name: "
8     read name
9     if [[ (-z $name) || ($name = 0) ]]; then
10        echo "bye"
11        stdout=1
12    elif [[ -n $name ]]; then
13        while [[ $stdout != 1 ]]; do
14            echo "enter your age: "
15            read age
16            if [[ ($age -eq 0) || (-z $age) ]]; then
17                echo "bye"
18                stdout=1
19            elif [[ $age -le $child ]]; then
20                echo "$name, your group is child"
21            elif [[ $age -gt $adult ]]; then
22                echo "$name, your group is adult"; else
23                if [[ ($age -ge 17) && ($age -le 25) ]]; then
24                    echo "$name, your group is youth"; fi
25            fi ;break
26        done ;fi
27    done
```

Следующий шаг

Решить снова

Рисунок 3.15: Скрипт возрастной группы – решение

Листинг:

```
child=16
adult=25
stdout=0

while [[ $stdout != 1 ]]
do
    echo "enter your name: "
    read name
    if [[ (-z $name) || ($name = 0) ]] ;then
        echo "bye"
        stdout=1
    elif [[ -n $name ]]; then
        while [[ $stdout != 1 ]] ;do
            echo "enter your age: "
            read age
            if [[ ($age -eq 0) || (-z $age) ]] ;then
                echo "bye"
                stdout=1
            elif [[ $age -le $child ]] ;then
                echo "$name, your group is child"
            elif [[ $age -gt $adult ]] ; then
                echo "$name, your group is adult" ;else
                if [[ ($age -ge 17) && ($age -le 25) ]] ;then
                    echo "$name, your group is youth" ;fi
                fi ;break
            done ;fi
        done
    done
```

3.4 Выполнение 3.4. Скрипты на bash: разное

Увеличить переменную «a» на значение «b» можно с помощью «let a=a+b», «let»a = a + b», «let «a+=b»» и «let»a=a+b». Прямое присваивание»a=a+b» без «let» лишь соединяет строки (рис. 3.16).

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b`? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15. Выберите все подходящие варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании – не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и пробелы, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

✓ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `a=$a+$b`
- ☒ `let a=a+b`
- ☒ `let "a = a + b"`
- ☒ `let "a+=b"`
- ☒ `let "a=$a+$b"`

Следующий шаг Решить снова

Рисунок 3.16: Увеличение переменной

Скрипт, содержащий «cd /home/bi/» и «echo»pwd»“, выводит»/home/bi», так как подстановка команды в обратных кавычках выполняется после смены каталога (рис. 3.17).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [programs.sh](#).

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

☒ Отлично!

- ☐ ``pwd``
- ☐ `/home/bi/Documents`
- ☐ `pwd`
- ☐ Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибок)
- ☒ `/home/bi`

Следующий шаг

Решить снова

Вы получили: 1 балл
Мои решения

Верно решил 35 241 учащийся
Из всех попыток 51% верных

Рисунок 3.17: Вывод текущего каталога в скрипте

Для проверки кода возврата программы, которая пишет в `stdout`, нужно сначала выполнить её, затем `if [[ $? -eq 0 ]]` или же создать условие `if «program > some_file.txt»`(рис. 3.18).

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if "program options arguments"` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if "pwd"`.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если её код возврата равен 0? Выберите **все верные** утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно **косая кавычка** (`'`), а не обычная (`"`) или двойная (`"`).

Выберите все подходящие ответы из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить свое решение с другими на [форуме решений](#).

- ☐ Ничего сделать нельзя
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`
- ☐ `if [[ "program" -eq 0 ]]`
- ☒ `if "program > some_file.txt"`
- ☐ Сначала `var="program"`, затем `if [[ $var -eq 0 ]]`

Следующий шаг

Решить снова

Рисунок 3.18: Возврат внешней программы

Функция «`counter`» с локальной переменной «`c1`» не изменяет глобальную переменную с тем же именем. После десяти вызовов «`c2`» становится равной 110,

а «c1» остаётся пустой, поэтому «echo» выводит «counters are and 110» (рис. 3.19).

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из bash-скрипта:

```
counter () # takes one argument
{
    local let "c1+=${1}"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 😊). В этом случае имеет смысл написать небольшой скрипт на bash, который сделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст

✓ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Следующий шаг Решить снова

Рисунок 3.19: Локальные переменные в функции

Написан скрипт для вычисления НОД с помощью рекурсивной функции «gcd», реализующей алгоритм Евклида. Скрипт ожидает ввода двух натуральных чисел, выводит результат и завершается по пустой строке словом «bye» (рис. 3.20).

Напишите скрипт на `bash`, который будет искать наибольший общий делитель (**НОД**, greatest common divisor, GCD) двух чисел.

При запуске ваш скрипт не должен ничего писать на экран, а просто ждет ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные – см. пример в видеофрагменте). После ввода чисел скрипт считает их НОД и выводит на экран сообщение **"GCD is <посчитанное значение>"**, например, для чисел 15 и 25 это будет **"GCD is 5"**. После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран **"bye"** и закончить свою работу.

Вычисление НОД несложно реализовать с помощью [алгоритма Евклида](#). Вам нужно написать функцию `gcd`, которая принимает на вход два аргумента (назовем их **M** и **N**). Если аргументы равны, то мы нашли НОД – он равен **M** (или **N**), нужно выводить соответствующее сообщение на экран (см. выше). Иначе нужно сравнить аргументы между собой. Если **M** больше **N**, то запускаем ту же функцию `gcd`, но в качестве первого аргумента передаем (**M-N**), а в качестве второго **N**. Если же наоборот, **M** меньше **N**, то запускаем функцию `gcd` с первым аргументом **M**, а вторым (**N-M**).

Пример корректной работы скрипта:

```
./script.sh
10 15
GCD is 5
7 3
GCD is 1
bye
```

Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.ч. смело вызывайте `gcd` прямо внутри `gcd`!

Примечание 2: для завершения работы функции в произвольном месте, можно использовать инструкцию `return` (все инструкции функции после `return` выполняться не будут). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в данной задаче можно обойтись и без использования `return`!

Подсказка: в случае проблем с решением задачи, обратите внимание на наши [рекомендации по написанию скриптов](#).

Рисунок 3.20: Задание на НОД

Напишите программу. Тестируется через `stdin → stdout`

✓ Здорово, всё верно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2
3 while [ true ]
4 do
5     read n1 n2
6     if [ -z $n1 ]; then
7         echo "bye"
8         break
9     else
10        gcd () {
11            remainder=1
12            if [ $n2 -eq 0 ]
13            then
14                echo "bye"
15            fi
16            while [ $remainder -ne 0 ]
17            do
18                remainder=$((n1%n2))
19                n1=$n2
20                n2=$remainder
21            done
22        }
23        gcd $1 $2
24        echo "GCD is $n1"
25    fi
26 done
```

Следующий шаг

Решить снова

Рисунок 3.21: Решение НОД

Листинг

```
#!/bin/bash
```

```

while [ true ]
do
    read n1 n2
if [ -z $n1 ]; then
    echo "bye"
    break
else
    gcd () {
        remainder=1
        if [ $n2 -eq 0 ]
        then
            echo "bye"
        fi
        while [ $remainder -ne 0 ]
        do
            remainder=$((n1%n2))
            n1=$n2
            n2=$remainder
        done
    }
    gcd $1 $2
    echo "GCD is $n1"
fi
done

```

Реализован калькулятор на bash, поддерживающий операции «+», «-», «*«,»/«,»%«,»***» и команду «exit». При некорректном вводе выводится «error», и скрипт завершается (рис. 3.22).

Напишите **калькулятор** на bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово **"exit"**. В этом случае скрипт должен вывести на экран слово "bye" и завершить работу.
2. **Три аргумента через пробел** – первый операнд (целое число), операция (одна из "+", "-", "*", "/", "%", "**") и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого переходим в режим ожидания новой команды.
3. **Любая другая команда** из одного аргумента или из трех аргументов, но с операцией не из списка. В этом случае нужно вывести на экран слово **"error"** и завершить работу.

Чтобы проверить работу скрипта, вы можете записать сразу несколько команд в файл и передать его скрипту на stdin (т.е. выполнить `./script.sh < input.txt`). В этом случае он должен вывести сразу все ответы на экран.

Например, если входной файл будет следующего содержания:

```
10 + 1
2 ** 10
exit
```

то на экране будет:

```
11
1024
bye
```

Если же на вход поступит следующий файл:

```
3 - 5
2/10
exit
```

то на экране будет:

```
-2
error
```

т.к. вторая команда была **некорректной** (в ней всего один аргумент, т.к. нет пробелов между числами и операцией, а единственная допустимая команда из одного аргумента это "exit").

Подсказка: в случае проблем с решением задач, обратите внимание на [наши рекомендации по написанию скриптов](#).

Рисунок 3.22: Задание на калькулятор

Напишите программу. Тестируется через stdin → stdout

✔ Отлично!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2
3 while [[ True ]]
4 do
5     read num1 op num2
6     if [[ $num1 == "exit" ]]
7     then
8         echo "bye"
9         break
10    elif [[ *$num1* =~ "^[0-9]+$" && *$num2* =~ "^[0-9]+$" ]]
11    then
12        echo "error"
13        break
14    else
15        case $op in
16            "+") let "res = num1 + num2";;
17            "-") let "res = num1 - num2";;
18            "/" ) let "res = num1 / num2";;
19            "*" ) let "res = num1 * num2";;
20            "%" ) let "res = num1 % num2";;
21            "**") let "res = num1 ** num2";;
22            *) echo "error" ; exit ;;
23        esac
24        echo "$res"
25    fi
26 done
```

Следующий шаг

Решить снова

Рисунок 3.23: Решение калькулятора

Листинг:

```
#!/bin/bash

while [[ True ]]
do
    read num1 op num2
    if [[ $num1 == "exit" ]]
    then
        echo "bye"
        break
    elif [[ *$num1* =~ "^[0-9]+$" && *$num2* =~ "^[0-9]+$" ]]
    then
        echo "error"
        break
    else
        case $op in
            "+") let "res = num1 + num2";;
            "-") let "res = num1 - num2";;
            "/" ) let "res = num1 / num2";;
            "**") let "res = num1 * num2";;
            "%") let "res = num1 % num2";;
            "**") let "res = num1 ** num2";;
            *) echo "error" ; exit ;;
        esac
        echo "$res"
    fi
done
```

3.5 Выполнение 3.5. Продвинутый поиск и редактирование

Команда «`find -iname»star“`” *находит больше файлов, чем «`find -name»star“`», так как игнорирует регистр. Из предложенного набора, которые найдет команда `find /home/bi -iname»star«`, но НЕ найдет команда `find /home/bi -name»star“` оказались «Star_Wars.avi» и «STARS.txt» (рис. 3.24).*

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -name "star*"` ?

Выберите все подходящие ответы из списка

✔ Отличное решение!

- ☐ stardust.mpeg
- ☐ star_trek OST.mp3
- ☒ Star_Wars.avi
- ☒ STARS.txt
- ☐ Eddard_Stark_biography.txt

Следующий шаг Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 30 903 учащихся
Из всех попыток 40% верных

Рисунок 3.24: `find` с `-iname` и `-name`

Опции «`-path`» и «`-name`» не всегда взаимозаменяемы: в одних случаях «`-name`» находит меньше файлов, в других — больше, в зависимости от шаблона, поэтому верные утверждения «В некоторых случаях `find` с `-name` найдет меньше файлов, чем `find` с таким же запросом, но с `-path`», «В некоторых случаях `find` с `-name` найдет больше файлов, чем `find` с таким же запросом, но с `-path`» (рис. 3.25).

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

✓ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Опции `-path` и `-name` всегда работают одинаково
- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- ☒ В некоторых случаях `find` с `-name` найдет меньше файлов, чем `find` с таким же запросом, но с `-path`
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☒ В некоторых случаях `find` с `-name` найдет больше файлов, чем `find` с таким же запросом, но с `-path`

Следующий шаг

Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 28 288 учащихся
Из всех попыток 26% верных

Рисунок 3.25: Сравнение `-path` и `-name`

При структуре `/home/bi/`, содержащей вложенные каталоги `«dir1»`, `«dir2»`, `«dir3»` с файлами `«file1»`, `«file2»`, `«file3»` соответственно, условие `«-mindepth 2 -maxdepth 3»` отсекает файлы, лежащие непосредственно в указанной директории (глубина 1). Под критерий попадают только файлы, расположенные на глубине 2 или 3. Таким образом, будут найдены `«file1»` и `«file2»`, но не `«file3»` (рис. 3.26).

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
/home/bi/
├── dir1
│   ├── file1
│   ├── dir2
│   │   ├── file2
│   │   └── dir3
│   │       └── file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

✓ Верно.

- ☐ Все кроме `file2`
- ☐ Только `file1`
- ☒ Все кроме `file3`
- ☐ Ни один файл найден не будет
- ☐ Только `file3`

Следующий шаг

Решить снова

Рисунок 3.26: `mindepth` и `maxdepth` в `find`

При добавлении контекста (`«-A»`, `«-B»`, `«-C»`) к поиску слова, которое

присутствует в каждой строке файла, размер результирующего файла не изменяется. Во всех четырёх случаях «results.txt» будет одинакового размера (рис. 3.27).

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в **каждой строке есть слово "word"**. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

✔ Отлично!

- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☐ `grep -C 1 "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях
- ☐ `grep -A 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`

Следующий шаг Решить снова

Рисунок 3.27: сравнение размеров вывода `grep` с контекстом

Регулярное выражение «`[xklXKL]?[uU]buntu$`» описывает строки, оканчивающиеся на «buntu» с возможной предшествующей буквой из набора. Единственная подходящая строка – «I prefer Kubuntu» (рис. 3.28).

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xklXKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

✔ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ I prefer Kubuntu
- ☐ Lubuntu is better than Windows
- ☐ Well, xubuntu is OK
- ☐ Kubuntu
- ☐ Mac OS X 10.9, Windows XP, Ubuntu 12.04
- ☐ Uuuubuntu!

Следующий шаг Решить снова

Рисунок 3.28: `grep` с регулярным выражением

Если в «`sed`» опустить опцию «`-n`», каждая строка, попавшая под шаблон, будет

напечатана дважды: один раз из-за автоматического вывода, второй – по команде «р» (рис. 3.29).

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Что произойдет, если в команде `sed -n "[a-z]*p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка

✔ Отлично!

- ☒ Каждая строчка будет выведена два раза
- ☐ Будут выведены все строки файла `text.txt`, в которых есть только большие буквы латинского алфавита
- ☐ На экран будет выведено всё содержимое файла `text.txt`
- ☐ На экран ничего не напечатается

Следующий шаг Решить снова

Вы получили: 1 балл
Мои решения

Верно решил 29 891 учащийся
Из всех попыток 40% верных

3 учащимся понравился этот шаг, а вам?

Следующий шаг >

Рисунок 3.29: sed без -n

Для замены «аббревиатур» (две и более заглавные латинские буквы) на слово «abbreviation» в файле «input.txt» с записью результата в «edited.txt» использована команда «sed „s/[A-Z]{2,}/abbreviation /g“ input.txt > edited.txt» (рис. 3.30).

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Запишите в форму ниже инструкцию `sed`, которая заменит все «аббревиатуры» в файле `input.txt` на слово «abbreviation» и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что в инструкции должны быть указаны и сам `sed`, и оба файла!

Под «аббревиатурой» будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте не может быть две «аббревиатуры» подряд. Например, текст `" You You and YOU !"` является некорректным (в нем есть две «аббревиатуры», но они идут подряд) и на таких примерах мы проверять вашу инструкцию не будем.

Пример: если у вас был текст `"Hi, I heard these songs by ABBA, TLA and DM !"`, то он должен быть преобразован в `"Hi, I heard these songs by ABBA, abbreviation and abbreviation !"`.

Примечание: после вашей замены «аббревиатуры» на слово «abbreviation» количество пробелов в тексте не должно меняться!

Внимание! Во время проверки мы не запускаем команду, которую вы ввели на реальном файле с «аббревиатурами» (это небезопасно, можно же ввести `rm -rf /*`)! Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем запускаем её смысловую часть (т.е. поиск по регулярному выражению и замена на «abbreviation») на тестовых примерах. К сожалению, наш запуск не идеально повторяет `sed`, но он очень близок к нему. Главная «несовместимость» заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. `*`, `+`, `?` и `{}`). Однако эту «несовместимость» легко исправить указав при помощи «(» и «)» какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве «a») нужно записать как `(a+)?` (при этом запись `(a)+?`, конечно же, не поможет).

Рисунок 3.30: Замена аббревиатур – условие

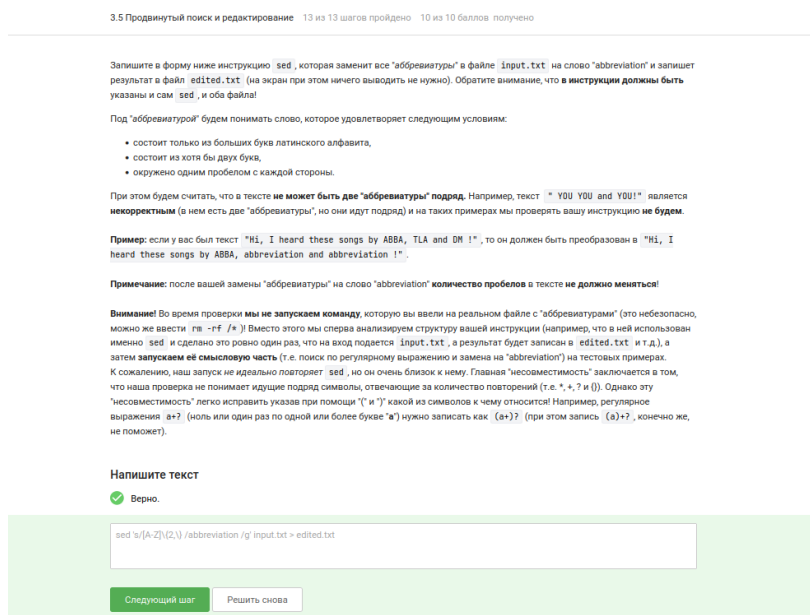


Рисунок 3.31: Замена аббревиатур – решение

3.6 Выполнение 3.6. Строим графики в gnuplot

Для того чтобы графики не закрывались при выходе из gnuplot, используется опция «-p» или «-persist» (рис. 3.32).

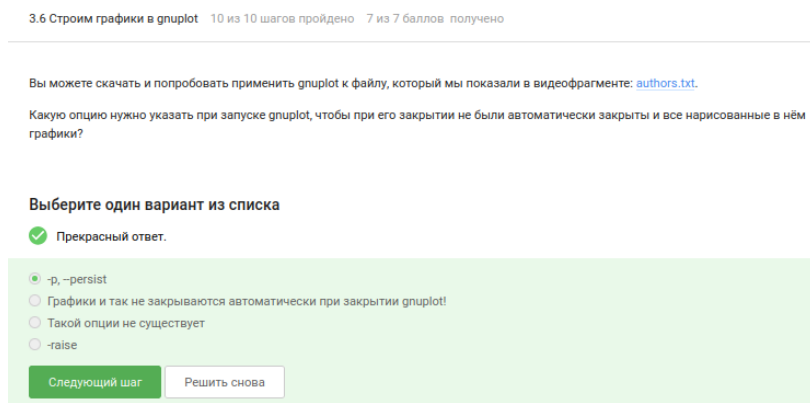


Рисунок 3.32: Сохранение графиков после закрытия gnuplot

Когда «data.csv» не содержит заголовка, а задано «set key autotitle columnhead», первая строка используется как название кривой (второй столбец) и исключается

из данных, поэтому на графике отображается 9 точек (рис. 3.33).

3.6 Строим графики в gnuplot 10 из 10 шагов пройдено 7 из 7 баллов получено

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один вариант из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название "polame", нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название – первое значение из второго столбца, нарисовано 10 точек
- ☐ Название "data.csv" using 1:2", нарисовано 10 точек

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл
[Мои решения](#)

Верно решили 27 489 учащихся
Из всех попыток 34% верных

Рисунок 3.33: `autotitle columnhead` без заголовка

Для размещения на оси ОХ только трёх заданных делений с текстовыми метками используется команда «`set xtics`». Строки конкатенируются с помощью оператора «`.`». Правильный ответ: «`set xtics („point 1, value“.x1 x1, „point 2, value“.x2 x2, „point 3, value“.x3 x3)`» (рис. 3.34).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [plot.gnu](#), [plot_advanced.gnu](#), [plot_advanced2.gnu](#). Все три скрипта основаны на [этой заметке](#), данные также взяты оттуда.

Предположим, что вы пишете `gnuplot`-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде **"point <номер точки>, value <значение соответствующей переменной>**". Например, для `x1=8`, `x2=18`, `x3=28`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20. Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно**!

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин *конкатенация*, который важен для выполнения данного задания. Под *конкатенацией* обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст

✓ Верно.

```
set xtics ("point 1, value "x1 x1, "point 2, value "x2 x2, "point 3, value "x3 x3)
```

Следующий шаг

Решить снова

Рисунок 3.34: Установка пользовательских `xtics`

Файл «`move.rot`» был изменён для зеркального отражения («`splot -x2-y2`»), вращения в обратную сторону («`zrot=(zrot+350)%360`») и удвоения скорости («`pause 0.1`»). Это позволило получить требуемую анимацию (рис. 3.35).

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять и удалять инструкции нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу gnuplot и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в gnuplot работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Напишите текст

✓ Всё получилось!

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Следующий шаг

Решить снова

Рисунок 3.35: Модифицированный move.rot

3.7 Выполнение 3.7. Разное

Для превращения прав «r-r-r-» в «rwxrwx-r-» подходят последовательности «chmod u+wx file.txt; chmod g+w file.txt», «chmod ug+w file.txt; chmod u+x file.txt», «chmod 764 file.txt», «chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt» (рис. 3.36).

Какая команда(ы) установят файлу `file.txt` права доступа `rw-rw-r--`, если изначально у него были права `g-r--r--`. Укажите **все верные варианты ответа!**

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнились все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите все подходящие ответы из списка

☒ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☒ `chmod ug+w file.txt; chmod u+x file.txt`
- ☐ `chmod u-wx file.txt; chmod g-w file.txt`
- ☒ `chmod 764 file.txt`
- ☒ `chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt`
- ☐ `chmod o-wx file.txt; chmod g-x file.txt; chmod a+wx file.txt`

Следующий шаг

Решить снова

Вы получили: 1 балл

[Мои решения](#)

Верно решили 24 989 учащихся

Из всех попыток 21% верных

Рисунок 3.36: Изменение прав доступа

Чтобы пользователь из группы «group» мог создавать файлы в каталоге «dir», принадлежащем root, можно использовать: «`sudo chmod o+w dir`» (добавляет право записи для «остальных» пользователей, к которым относится и user), «`sudo chown user:group dir`» (меняет владельца и группу каталога на user и group, после чего user получает полный контроль как владелец), «`sudo chmod a+w dir`» (даёт право записи всем, что разрешает user создавать файлы), «`sudo chown user dir`» (меняет только владельца на user, после чего user как владелец может записывать файлы в каталог) (рис. 3.37).

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rw-r--r--` (владелец `root`, группа `root`). Таким образом никто кроме пользователя `root` не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды `user` из группы `group` всё-таки сможет создать файл внутри `dir`? Укажите **все верные** варианты ответов!

Примечание: считаем, что все команды выполняются от имени `user`, если явно не указано, что команда выполнена с `sudo`.

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно.

Дело в том, что если создать при помощи `sudo` файл с правами `rw-r--r--` в директории, которая принадлежит пользователю, то возникнет любопытная ситуация. С одной стороны пользователь может удалить этот файл (т.к. ему разрешено удалять **все** файлы внутри его директории) и может прочитать его содержимое (т.к. право `r` у файла установлено для всех), с другой стороны он не может этот файл редактировать (т.к. право `w` у файла есть только для `root`). При этом некоторые "умные" редакторы, например, `vim` позволяют даже редактировать этот файл, но сделают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалять мы можем, а раз можем читать, то и копию создать не сложно). Итого получается, что несмотря на права `rw-r--r--`, пользователь может сделать с этим файлом почти всё что угодно!

В случае же, когда речь идет о директории созданной `root`, ситуация будет проще: пользователь сможет посмотреть её содержимое (у него есть право `r`), но удалять и создавать файлы в ней не сможет (права `w` у него нет).

Важно отметить, что директории в `Linux` это в каком-то смысле файлы. Содержимое такого "файла" – это записи о файлах и поддиректориях этой директории (грубо говоря их названия). Таким образом, право `r` у директории дает возможность просматривать "записи", т.е. просматривать её состав. Право `w` у директории дает возможность удалять/добавлять новые "записи", т.е. удалять/создавать файлы/поддиректории в ней.

На самом деле и это еще не всё. Существует так называемый `sticky bit` (атрибут файла или директории), выставление которого меняет описанное выше поведение. Файлы (или директории) с таким атрибутом сможет удалить только их владелец вне зависимости от прав, установленных у директории, в которой эти файлы (или директории) лежат!

Отдельное спасибо слушателю курса [Alexey Antipovsky](#) за помощь в оформлении **Примечания 2!**

Выберите все подходящие ответы из списка

☒ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `sudo chmod o+w dir`
- ☒ `sudo chown user:group dir`
- ☒ `sudo chmod a+w dir`
- ☐ `chmod o+w dir`
- ☐ `sudo chmod o+x dir`
- ☒ `sudo chown user dir`

Следующий шаг

Решить снова

Вы получили: 1 балл

[Мои решения](#)

Верно решили 22 823 учащихся
Из всех попыток 17% верных

Рисунок 3.37: Предоставление доступа к чужой директории

Утилита «wc» способна подсчитывать строки, слова, символы и длину самой длинной строки, но не количество конкретных букв (рис. 3.38).

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

☒ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Количество определенных букв (например, количество букв "А")
- ☒ Количество символов
- ☒ Количество строк
- ☒ Длину самой длинной строки
- ☒ Количество слов

Следующий шаг

Решить снова

Рисунок 3.38: Возможности `wc`

Размер текущей директории в человеко-читаемом формате выводится командой `«du -h -s»` (или `«du -sh»`). Ключ `«-h»` включает удобные единицы измерения, `«-s»` подводит итог (рис. 3.39).

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо `2848 байт` надо вывести `2.8K`) и **больше** на экран выводить **ничего не нужно**). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по `888 Кбайт` и две поддиректории в каждой из которой лежит по файлу в `488 Кбайт`, то загаданная команда должна вывести на экран одно число: `2.4K` (также на экране может быть выведен еще и символ `..`, обозначающий, что это размер именно текущей директории).

Напишите текст

☒ Отлично!

`du -h -s`

Следующий шаг

Решить снова

Рисунок 3.39: Вывод размера директории

Наиболее короткая команда для одновременного создания трёх поддиректорий `«dir1»`, `«dir2»`, `«dir3»` : `«mkdir dir{1..3}»` (рис. 3.40).

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✓ Правильно.

`mkdir dir{1..3}`

Следующий шаг

Решить снова

Рисунок 3.40: Кратчайшее создание нескольких каталогов

4 Выводы

В ходе выполнения третьего этапа курса «Введение в Linux» были освоены продвинутые приёмы работы в vim, написание сценариев на bash с условными переходами и циклами, использование функций, регулярные выражения в grep и sed, построение и анимация графиков в gnuplot, а также ряд полезных утилит (wc, du, chmod)

5 Список литературы

1. Курс «Введение в Linux» на платформе Stepik [Электронный ресурс] URL:
<https://stepik.org/course/73/>